

Interview Notes — Trek Ready App

Interview Notes — Trek Ready App

Personal reflection on why I built this, how it evolved, and what it taught me.

Technical reference is in CLAUDE.md. Bug detail is in DEBUG_LOG.md.

Tech stack

Language & runtime

Component	Choice	Version
Language	Python	3.9.6
Runtime	CPython (macOS)	3.9.6

Core framework

Library	Role	Version
Streamlit	UI framework — reactive web app without frontend code	1.50.0
streamlit-folium	Renders Folium maps as Streamlit components	0.25.3

Data & visualisation

Library	Role	Version
Pandas	DataFrames for chart data and tabular display	2.3.3
Altair	Declarative charts — risk heatmap, elevation profile, readiness gauge	5.5.0
Folium	Interactive Leaflet.js maps via Python	0.20.0

AI & external APIs

Service / Library	Role	Key
Anthropic SDK	Optional AI narrative layer (risk summary, recommendations, briefing)	Paid — requires API key

Service / Library	Role	Key
Open-Meteo API	Daily weather forecast (temperature, precipitation, wind)	Free — no key required
Overpass API (OSM)	Real GPS trail geometry and route name lookup	Free — no key required
Wikipedia API	Trail descriptions, distance, duration, difficulty (text extraction)	Free — no key required

Export & environment

Library	Role	Version
ReportLab	PDF generation — expedition briefing and interview notes	4.5.1
python-dotenv	Loads API keys from .env file; override=True prevents Claude Code key conflict	1.2.1

Testing & CI

Tool	Role	Version
pytest	Unit test runner — 262 tests across 16 test files	8.4.2
GitHub Actions	CI — runs full test suite on push and pull request	N/A

Deployment target

Platform	Details
Streamlit Community Cloud	Free hosting; deploys from GitHub; secrets managed via dashboard
Local dev	streamlit run app.py; hot-reloads on file save

Notable stdlib modules used (no extra install)

- **urllib.request** — all HTTP calls to Open-Meteo, Overpass, and Wikipedia (no requests library needed)
- **re** — text parsing for Wikipedia extract (distance, elevation, duration, difficulty, season)
- **json** — API response parsing
- **pathlib** — file I/O for export
- **smtplib / email** — optional email sending script

- `**from __future__ import annotations**` — Python 3.9 compatibility for X | None type hints
-

Why I built this

Trip planning for multi-day treks is surprisingly fragmented. You're cross-referencing packing lists from blogs, asking Reddit for gear advice, checking weather forecasts, reading park permit rules, and doing rough math on daily distances — all in different tabs and never synthesised into one view.

I wanted a single place that could take a route and a profile (experience level, dates, trip style) and produce a coherent picture: how ready am I for this, what are the actual risk factors, what do I pack, and what should each day look like. The goal was a planning tool that felt like advice from an experienced trekking guide — specific, contextual, and honest about where the uncertainty is.

The secondary goal was to build it with AI assistance and pay attention to *how* that collaboration actually works in practice — not just what gets built, but what the working patterns are, where the AI earns its keep, and where you still have to think hard yourself.

Key decisions and tradeoffs, phase by phase

Phase 1–2: Streamlit as the foundation

Decision: Use Streamlit over Flask/Django.

Tradeoff: Streamlit's reactivity model (full rerun on every widget change) is unusual and creates state management challenges. But the data app pattern it encourages — inputs in the sidebar, outputs in the main area — matched the planning tool UX perfectly. The alternative would have been weeks of frontend work before I had anything to look at.

Would I change it? No for a solo tool like this. For a product with many users or complex interactions, I'd probably move to a proper frontend.

Phase 3: Rules-based scoring over ML/LLM inference

Decision: Five deterministic scoring dimensions with explicit thresholds, not a language model.

Tradeoff: A rules-based model is transparent and debuggable — you can explain exactly why a score changed. But it can't generalise beyond the rules you've written. A Beginner doing a 4215m trail gets penalised correctly, but the model doesn't understand that acclimatisation protocol matters more than raw elevation.

What this forced: The rules had to be explicit and defensible. That turned out to be useful — every tradeoff got documented and every threshold got a test.

Would I change it? I'd still start rules-based. An LLM could generate a narrative *about* the scores (which is exactly what `ai_layer.py` does), but using it to compute the scores themselves would make the system opaque in a context where interpretability matters for safety.

Phase 5: Sidebar nav → `st.tabs`

Decision: Replace sidebar radio-button navigation with tabs.

What prompted it: The sidebar nav created a physical disconnect between the control and the content it switched. It also generated several hard-to-reproduce state bugs where the nav could get stuck. Tabs place the navigation immediately above the content it reveals.

Tradeoff: Tabs eliminated query-param page routing entirely, which simplified the codebase significantly (removed a whole helper class). The cost was losing the ability to deep-link to a specific tab via URL — though `?focus=` for the guidance section was preserved.

Phase 6b: Two-pass gear risk design

Decision: Compute initial risk → build packing plan → adjust gear risk based on plan quality → recompute readiness.

Tradeoff: The packing plan uses risk scores to decide what to include, which creates a circular dependency if you try to let the plan influence the scores that drive it. The two-pass approach breaks the cycle cleanly: the initial scores drive the plan, then the plan's quality feeds back into the scores as a one-time adjustment. It's slightly non-intuitive (gear risk changes after you see the packing list) but it's fully deterministic and testable.

Phase 8a: Auto-deriving weather from forecast

Decision: Replace the manual "Expected weather" dropdown with a value auto-derived from the Open-Meteo forecast, with manual override preserved.

What changed the thinking: Once trip start date and route coordinates were both in the sidebar, the manual weather field became redundant for trips within the 16-day forecast window. Keeping it as a manual field for out-of-window trips (future planning) still made sense, so the field became conditional rather than being removed.

Tradeoff: This means changing the route or date resets the weather field, which could surprise a user who had manually overridden it. Documented this in the UX but it's a real friction point.

Phase 8b: Route discovery via OSM + Wikipedia

Decision: Two-source discovery (Wikipedia for text/stats, Overpass for GPS coordinates) rather than a single commercial API.

Tradeoff: Both sources are free with no API keys, which matters for a publicly deployable app. The cost is that data quality is uneven — some trails have rich OSM geometry, others have nothing. Wikipedia extracts are useful but text-parsing regex is fragile. The result is a "discovered" profile that's clearly labelled as approximate, not presented as authoritative.

What I learned: Wikipedia `opensearch` handles partial/fuzzy queries far better than Overpass name search, which times out on global scans. The right approach ended up being: Wikipedia for discovery and naming, Overpass bbox for GPS geometry.

How the rules engine works — and its limits

The five scoring dimensions

The core of the app is ``scoring_model.py``. Each trip is evaluated across five independent dimensions, each scored 0–20. The final readiness score is 100 minus a weighted average of those five scores, expressed as a percentage.

1. Fitness risk scores how physically demanding the route is relative to the hiker's experience. The key design choice was scoring against **daily** distance and elevation (total divided by moving days), not trip totals — because 80 km over 10 relaxed days is genuinely different from 80 km in 3 hard days. Thresholds are:

- Daily distance: ≤ 10 km (+0), ≤ 15 km (+2), ≤ 20 km (+5), ≤ 25 km (+8), > 25 km (+10)
- Daily elevation gain: ≤ 300 m (+0), ≤ 600 m (+2), ≤ 1000 m (+4), ≤ 1500 m (+6), > 1500 m (+8)
- Experience modifier: Beginner (+4), Intermediate (+1), Advanced (–5)

2. Altitude risk scores exposure to altitude-related hazard based on the route's maximum elevation. The zero-risk threshold was deliberately set at 800 m (not 1200 m as originally coded) because mountain environments below 1200 m still carry meaningful cold and weather risk. Thresholds: < 800 m (+0), 800–1500 m (+3), 1500–2500 m (+7), 2500–3500 m (+12), ≥ 3500 m (+16), with experience modifiers of +3/+1/–4.

3. Weather risk scores the environmental baseline plus the user's expected conditions. Originally this used month names parsed from the route's ``best_season`` field to infer a season — which was hemisphere-blind and broke for Patagonia. The fix was to replace it entirely with the route's ``climate`` field (Cold/Mountain +4, Tropical/Desert +3, Temperate +1), then add weather condition (Mixed +3, Unstable +6) and a regional bonus for Patagonia/alpine areas (+2).

4. Logistics risk scores operational complexity: remoteness (High +6, Moderate +3, Low +1), permit requirement (+3), transport complexity (High +4, Moderate +2), lodging tier (Hotel +0 through Campsite +4), and self-guided (+2). The lodging tier ordering was the source of a bug — the original ``else`` branch caught both Hotel and Campsite and penalised them equally. Making each tier explicit fixed it.

5. Gear risk scores how demanding the kit requirements are: climate (Cold/Mountain +4, other +2–3), lodging tier (same scale as logistics), pack weight preference (Minimal +5, Light +3, Balanced +1, Heavy +2), and expected weather (Mixed +2, Unstable +4).

How the five scores combine into a readiness percentage

...

```
readiness = 100 – weighted_average(dimension_scores / 20 × 100)
```

...

Weights: Fitness 1.1, Altitude 1.0, Weather 1.0, Logistics 0.9, Gear 0.8. Fitness is weighted highest because it's the most direct predictor of trip difficulty for the individual; gear is lowest because good packing can

compensate more readily than fitness can. The result is inverted — a higher risk sum means lower readiness.

A score of 70+ is labelled Low risk, 55–69 Moderate, 40–54 High, 25–39 Very High, <25 Extreme.

Post-scoring adjustments

Two adjustments are applied after the initial scores:

Guided trip reduction: For guided trips, Fitness risk is reduced by 2 (the guide manages pace and turnaround decisions) and Gear risk is reduced by 2 (the guide carries group safety equipment). These only apply to guided trips and do not touch Altitude, Weather, or Logistics.

Packing plan feedback: After the packing plan is built, if it has no warnings (weight within preference, critical items present, shelter covered if camping), Gear risk is reduced by 3 points. This creates a mild incentive loop — better packing preparation directly improves your readiness score, which feels correct behaviourally.

What the model explicitly does not do

- ****It doesn't score acclimatisation rate.**** Whether you're ascending 500 m/day or 2000 m/day doesn't affect Altitude risk — only where you end up. This was a deliberate choice to avoid double-counting with Fitness risk, which already scores daily elevation gain.
- ****It doesn't model weather variability.**** Unstable weather is penalised (+6) but the model treats every Unstable day the same. It doesn't know that Patagonia in October is genuinely changeable vs. a mildly cloudy lowland route.
- ****It doesn't learn.**** Every threshold is hardcoded. If the thresholds are wrong for a new type of route (e.g. desert ultramarathon), the scores will be off and there's no feedback mechanism to correct them.
- ****It doesn't account for group dynamics.**** Two experienced hikers supporting each other is meaningfully safer than one solo, but the model treats both the same.

These limitations are acceptable for a planning tool that explicitly labels itself as a readiness estimate, not a safety guarantee. The scoring is transparent enough that a user can see exactly why their score is what it is and adjust their preparation accordingly.

Scoring reference — full thresholds per dimension

Each dimension scores 0–20. Scores from sub-factors are summed, then clamped to the 0–20 range.

Fitness risk

Daily distance (total km ÷ moving days)

Daily distance	Points
≤ 10 km	+0
11–15 km	+2
16–20 km	+5
21–25 km	+8
> 25 km	+10

Daily elevation gain (total gain ÷ moving days)

Daily elevation gain	Points
≤ 300 m	+0
301–600 m	+2
601–1000 m	+4
1001–1500 m	+6
> 1500 m	+8

Experience modifier

Experience	Points
Beginner	+4
Intermediate	+1
Advanced	–5

Altitude risk

Maximum elevation of the route

Max elevation	Points
< 800 m	+0
800–1499 m	+3
1500–2499 m	+7
2500–3499 m	+12
≥ 3500 m	+16

Experience modifier

Experience	Points
Beginner	+3
Intermediate	+1
Advanced	−4

Weather risk

Route climate (from route data)

Climate	Points
Cold / Mountain / Alpine	+4
Tropical / Desert	+3
Temperate	+1

Expected weather condition (user input or auto-detected from forecast)

Condition	Points
Stable	+0
Mixed	+3
Unstable	+6

Regional bonus

Region	Points
Patagonia or Alpine	+2
All others	+0

Logistics risk

Remoteness

Remoteness	Points
High	+6
Moderate	+3
Low	+1

Permit required

Permit	Points
Yes	+3
No	+0

Transport complexity

Transport	Points
High	+4
Moderate	+2
Low	+0

Lodging type

Lodging	Points
Hotel	+0
Backcountry hut	+1
Refugio / Campsite	+2
Mixed camping / refugio	+3
Campsite	+4

Trip style

Style	Points
Guided	+0
Self-guided	+2

Gear risk

Route climate

Climate	Points
Cold / Mountain	+4
Hot	+3
Other	+2

Lodging type (same tier ordering as Logistics)

Lodging	Points
Hotel	+0
Backcountry hut	+1
Refugio / Campsite	+2
Mixed camping / refugio	+3
Campsite	+4

Pack weight preference

Pack preference	Points
Minimal	+5
Light	+3
Balanced	+1
Heavy	+2

Expected weather condition

Condition	Points
Stable	+0
Mixed	+2
Unstable	+4

Readiness score formula

...

$\text{readiness} = 100 - \text{weighted_average}(\text{dimension_score} / 20 \times 100)$

...

Dimension weights

Dimension	Weight
Fitness risk	1.1
Altitude risk	1.0
Weather risk	1.0

Dimension	Weight
Logistics risk	0.9
Gear risk	0.8

Fitness is weighted highest because it is the most direct predictor of individual difficulty. Gear is lowest because good preparation can compensate more readily than fitness can.

Risk label mapping

Readiness score	Label
≥ 70	Low
55–69	Moderate
40–54	High
25–39	Very High
< 25	Extreme

Post-scoring adjustments

Guided trip (applied after initial scoring, before packing)

Dimension	Adjustment
Fitness risk	–2
Gear risk	–2

Packing plan quality (applied after the packing plan is built)

Condition	Adjustment
No warnings (kit complete, weight within preference, shelter covered)	Gear risk –3
Any warnings present	No change

What I'd do differently

Start with tabs. The sidebar navigation experiment cost several phases of state bugs. ``st.tabs`` is the correct pattern for a multi-view app sharing a common context — it would have saved a lot of debugging.

Design the scoring model as a unified pipeline from day 1. The parallel scoring system in ``risk_engine.py`` (which could give a different answer from ``scoring_model.py``) was the most consequential

architectural mistake. The UI could show contradictory readiness and risk label numbers. The fix was straightforward once caught, but the damage to user trust if it had shipped publicly would have been significant.

Write integration tests earlier. The first wave of tests covered individual functions well but didn't catch when two correct functions combined incorrectly (e.g. fitness risk ignoring ``trip_profile``). An end-to-end test from route input to readiness score would have surfaced several bugs earlier.

Add ``base_elevation_m`` to route data from the start. The simulated elevation profile looked correct for simple routes but broke for high-gain loops like the TMB where ``total_gain > max_elevation_m``. A one-field addition to route data fixed it entirely — it was just never considered until a user reported the issue.

Be more explicit about data provenance throughout. The app mixes authoritative curated data (route library), computed estimates (scoring model), approximations (simulated elevation), and dynamically discovered data (OSM/Wikipedia). Surfacing which category a piece of data falls into would help users calibrate how much to trust it.

What I learned about AI-assisted development

Documentation as a first-class input. CLAUDE.md became the most important file in the project — not because it was useful to read, but because maintaining it forced precision about what had been decided and why. When I had to update it after a phase, I often caught ambiguities in the decisions I thought were settled. The doc was both an output and a forcing function.

AI is strongest on well-scoped, well-contexted tasks. "Add a weather integration using Open-Meteo" with a link to the API structure produced good code. "Make the app better" produces generic suggestions. The quality of the output scales with the specificity of the input — which is true of other engineers too, but the leverage is higher here.

Tests are the AI's fastest feedback loop. When a change broke a test, the AI could see the exact failure and fix it precisely. When a change broke something in the UI that wasn't tested, the feedback loop was much slower — it required restarting the app, navigating to the right page, and describing what looked wrong. The investment in 262 tests wasn't about coverage metrics; it was about shortening the iteration cycle.

The AI doesn't hold context across sessions. CLAUDE.md existed specifically because the AI couldn't remember why the scoring model was redesigned in Phase 6b, or why ``load_dotenv(override=True)`` was chosen over the default. Every non-obvious decision needed to be written down for future sessions to work from. This is a fundamentally different documentation discipline than writing docs for human readers.

Review generated code as you would a PR from a smart but unfamiliar contributor. The AI rarely makes obvious mistakes, but it will sometimes make architectural choices that are locally reasonable but globally inconsistent — like adding a new pattern that duplicates an existing helper, or writing a function that works but isn't testable because the logic is buried in the wrong place. Fast review before accepting generated code is cheaper than untangling it later.

The biggest wins were on tedious but well-defined work. Test generation, schema validation, repetitive data entry (the 5 routes with full field sets), migration of UI patterns across multiple day cards — these took seconds instead of hours. The big decisions (should we use rules or LLM? which API source for GPS data?) still required real thinking. The AI accelerated execution; it didn't replace judgment.

What's still left to do

- Integration test covering the full pipeline: route → scoring → packing → adjusted scores → itinerary → briefing
- Historical weather data for trips beyond the 16-day forecast window
- Route comparison view (side-by-side readiness scores for two configurations)
- Packing checklist export with checkboxes
- More routes in the curated library

See CLAUDE.md section 7 for the full backlog.

Next step: moving to React

Why I would move to React

Streamlit was the right choice to get here fast. The entire working app — scoring engine, weather integration, maps, PDF export, and 262 tests — was built without a single line of HTML or CSS. For a prototype and portfolio project, that speed-to-working-product tradeoff is correct.

For a real product with real users, Streamlit becomes a ceiling rather than an accelerator. The specific limitations that would drive a React migration:

Streamlit reruns the entire script on every widget change. Switching a dropdown, clicking a button, or changing a date triggers a full Python re-execution. For this app that means re-fetching weather, re-scoring the trip, and re-rendering every component on every interaction. React only re-renders what changed.

Layout and styling are fixed by Streamlit's component system. You can't control spacing, typography, or responsive behaviour precisely. The mobile experience is poor. Branding is impossible beyond the default Streamlit theme.

State management is a workaround, not a feature. Session state in Streamlit is a global dict that has to be carefully managed to avoid widget conflicts. React's component state, combined with libraries like Zustand or React Query, is designed for exactly this kind of problem.

Interactions that Streamlit can't do at all: drag-and-drop itinerary reordering, inline editing of day plans, animated transitions between views, optimistic UI updates, and complex form validation.

What the migration would look like

The key insight: **the Python logic stays completely unchanged**. The scoring model, route discovery, weather client, packing engine, and itinerary generator are all pure Python functions with no Streamlit dependency. They become a FastAPI backend with no rewriting — just adding API endpoints around the existing functions.

Phase 1 — Add a FastAPI layer (one to two weeks)

Wrap the existing Python modules in HTTP endpoints:

```
```python
from fastapi import FastAPI
from scoring_model import score_trip_risks, overall_readiness_score
from weather_client import fetch_forecast

app = FastAPI()

@app.post("/api/score")
def score(route: RouteIn, profile: ProfileIn, experience: str, weather: str):
 scores = score_trip_risks(route.dict(), profile.dict(), experience, weather)
 return {"scores": scores, "readiness": overall_readiness_score(scores)}

@app.get("/api/forecast")
def forecast(lat: float, lon: float, start_date: str, days: int):
 return fetch_forecast(lat, lon, date.fromisoformat(start_date), days)
```
```

Every module already has unit tests, so the API layer can be validated immediately. Run Streamlit and FastAPI side by side during transition — no big-bang cutover.

Phase 2 — Build the React frontend (two to four weeks)

Component mapping from the current Streamlit app:

| Streamlit component | React equivalent |
|-------------------------------------|---------------------------------------|
| st.sidebar + st.selectbox | Sidebar with controlled Select inputs |
| st.tabs | Tab component (Headless UI or Radix) |
| st.altair_chart (risk heatmap) | Recharts BarChart |
| st.altair_chart (elevation profile) | Recharts AreaChart |
| st.folium (trail map) | react-leaflet Map + Polyline + Marker |
| st.metric | Custom MetricCard component |

| Streamlit component | React equivalent |
|---------------------|-------------------------------|
| st.expander | Accordion (Radix UI) |
| st.download_button | fetch() + Blob + anchor click |
| st.spinner | Suspense or loading skeleton |

Phase 3 — State management

Replace Streamlit's session state with React Query for server state (scores, forecast, itinerary — all derived from API calls) and a small Zustand store for UI state (selected route, trip profile, active tab). React Query handles caching, background refetching, and loading states automatically — the weather forecast cache that required explicit `@st.cache_data(ttl=3600)` in Streamlit becomes a one-line `staleTime` config.

What would not change

- All Python business logic (scoring, packing, discovery, weather)
- The FastAPI endpoints once written
- The test suite — all 262 tests run unchanged
- The data model and route library
- Deployment target — FastAPI + React deploys to the same Streamlit Cloud alternatives (Railway, Render, Fly.io) or to any standard cloud platform

Estimated total migration effort: three to six weeks for a solo developer who knows React, working part-time. The backend is the hard part of this app and it's already done. The frontend is relatively thin — mostly forms, charts, and a map.